WEEK 1 · LESSON 3 OF 24 · HANDOUT

Lists, Dictionaries and Loops

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=W8KRzm-HUcc — open the lesson online to watch inside the course.

🎓 **Go deeper:** freeCodeCamp: Python for Beginners — full course — https://www.youtube.com/watch?v=eWRfhZUzrAc

Your goal in this lesson: Store many values in lists and dictionaries, and use `for` loops to process them one by one — the pattern behind all data work.

Lists: many values in order

```
daily_sales = [42000, 38500, 51000, 47200, 60300]   # RWF, Monday to Friday
print(daily_sales[0])     # first item -> 42000
print(daily_sales[-1])    # last item -> 60300
print(len(daily_sales))   # 5 days
print(sum(daily_sales))   # total for the week
print(max(daily_sales), min(daily_sales))
```

Python counts from **0**, so the first item is `[0]`. Negative numbers count from the end.

Slicing takes part of a list: `daily_sales[1:3]` gives items 1 and 2 (the end is not included).

Changing a list:

```
daily_sales.append(55800)   # add Saturday
daily_sales[0] = 43000      # correct Monday
print(daily_sales)
```

Dictionaries: values with names

A **dictionary** stores **key: value** pairs — like a real dictionary stores word: meaning.

```
prices = {"Rice 5kg": 7500, "Sugar 1kg": 1600, "Milk 1L": 800}
print(prices["Sugar 1kg"])         # 1600
prices["Cooking oil 1L"] = 3200    # add a new product
prices["Milk 1L"] = 850            # update a price
print(prices.keys())
print("Bread" in prices)           # False
```

Use a dictionary when you look things up **by name**, and a list when **order** matters.

for loops: do something for each item

```
for amount in daily_sales:
    print(amount)
```

The **indented** lines (4 spaces) run once for every item. Indentation is how Python knows what is inside the loop.

A very common pattern is the **running total**:

```
total = 0
for amount in daily_sales:
    total = total + amount
print(f"Week total: {total:,} RWF")
```

Looping over a dictionary:

```
for product, price in prices.items():
    print(f"{product}: {price:,} RWF")
```

`range()` gives numbers to loop over: `range(5)` gives 0, 1, 2, 3, 4.

```
for day in range(1, 8):
    print("Day", day)
```

List comprehensions: a short way to build lists

```
with_vat = [round(p * 1.18) for p in [7500, 1600, 800]]
print(with_vat)   # [8850, 1888, 944]
```

Read it as: *"for each p in the list, calculate p × 1.18"*. You will see this style often in data code.

Why this matters for AI

A dataset is really a big list of examples, and each example is like a dictionary of features: `{"age": 34, "sales": 180000, "repaid": True}`. Training a model is, underneath, loops over these examples many times. NumPy and Pandas (Week 2) will do these loops for you — much faster — but the idea is the same.

Key terms

- **List** — an ordered collection: `[1, 2, 3]`.
- **Index** — an item's position, starting at 0.
- **Dictionary** — key: value pairs: `{"Milk 1L": 800}`.
- **Loop** — code that repeats for each item.
- **Indentation** — the spaces that show which lines belong to a loop or block.
- **List comprehension** — a one-line way to build a new list from another.

Remember this: Lists keep values in order, dictionaries find values by name, and `for` loops process each value. Data science is built on these three ideas.

Name: _____ Date: _____

Activity

Task 1

Make a list of the marks of 6 students (out of 100). Print the highest, the lowest and the average (`sum(...) / len(...)`).

Task 2

Use a loop to count how many students scored 50 or more.

Task 3

Make a dictionary of 4 Rwandan districts and their population (look up rough numbers or invent them). Loop over it and print each district in a sentence.

Task 4

Use a list comprehension to convert a list of prices in RWF to USD (choose an exchange rate and note the date you used).

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does `[10, 20, 30][0]` give?
   - A. An error
   - B. 20
   - C. 30
   - D. 10
2. What does `[10, 20, 30][-1]` give?
   - A. 10
   - B. -1
   - C. An error
   - D. 30
3. When should you use a dictionary instead of a list?
   - A. When you look values up by name (key)
   - B. When order is the only thing that matters
   - C. Only for numbers
   - D. Never
4. How does Python know which lines are inside a for loop?
   - A. Indentation (spaces at the start of the line)
   - B. The word END
   - C. Capital letters
   - D. Curly brackets
5. What does `range(3)` give?
   - A. 1, 2, 3
   - B. 0, 1, 2
   - C. 3, 2, 1
   - D. 0, 1, 2, 3
6. What does `[x * 2 for x in [1, 2, 3]]` give?
   - A. [3, 4, 5]
   - B. 6
   - C. [2, 4, 6]
   - D. [1, 2, 3, 1, 2, 3]

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.